

WORKSHOP SERIES

Introduction to Neural Networks & Backpropagation

From linear models to deep learning optimisation

Recap from Last Workshop

Foundations

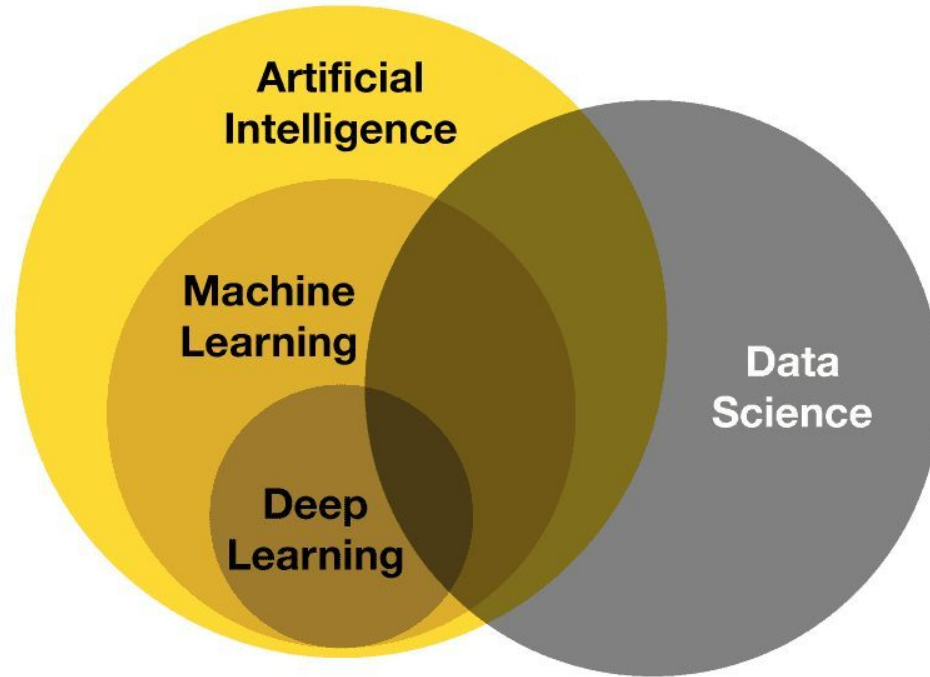
Introduced machine learning models, parameters, and loss functions.

Linear Regression

Covered linear regression and gradient descent optimisation.

Activation Functions

Introduced activation functions and why non-linearity is necessary.



Linear Regression & Neural Links

- **Simple linear regression formula: $y = mx + c$ (optimising 2 parameters to minimise loss).**
- **The core goal is to find a function that best fits a dataset.**
- **Neural networks share this exact goal just scaling up to many more parameters.**

Key Takeaway

Neural networks are generalised extensions of linear regression with interconnected layers and non-linear transformations.

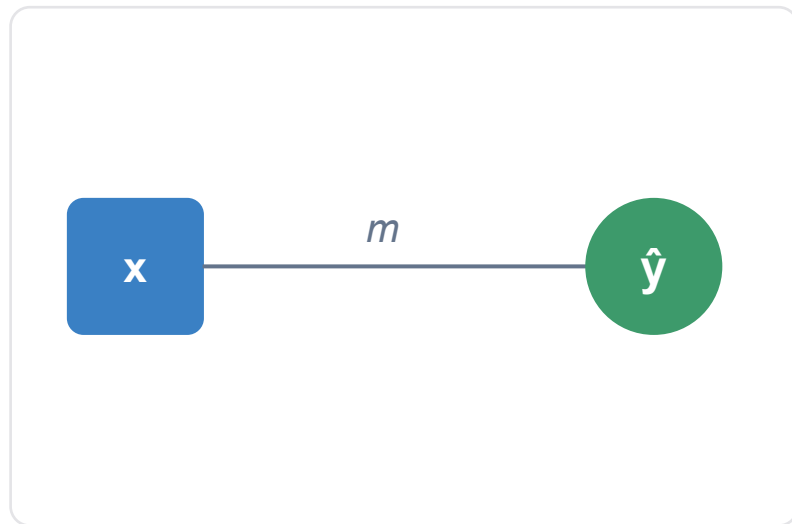
Single Neuron Architecture

What is the output prediction ($\hat{y}$)?

A single neuron with 1 input, 1 output, and 1 edge computes:

$$\hat{y} = mx + c$$

This is equivalent to simple linear regression.

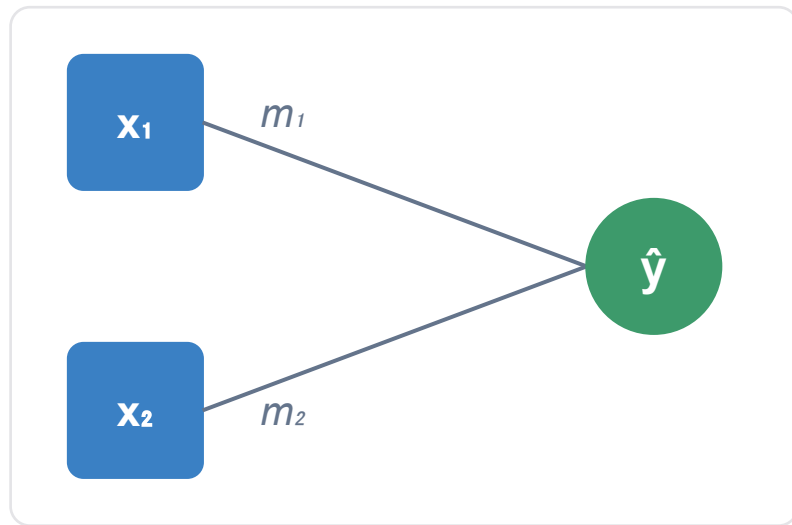


Scaling Inputs

What is the output of this expanded network?

$$\hat{y} = m_1x_1 + m_2x_2 + c$$

Notice that combining multiple linear inputs without an activation function still results in a linear model!

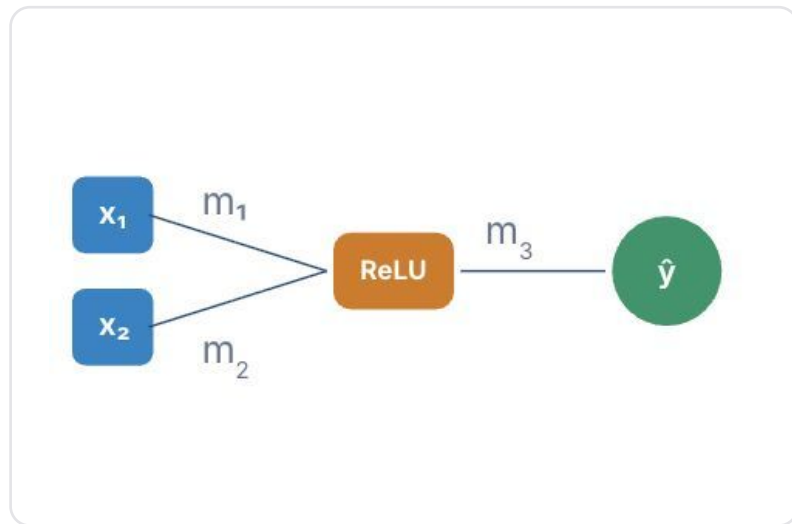


Adding Layers & Non-Linearity

Introducing Activation Functions

$$\hat{y} = m_3 \cdot \text{ReLU}(m_1 x_1 + m_2 x_2 + c_1) + c_2$$

By nesting linear transformations with non-linear activation functions (like ReLU), neural networks can model complex, non-linear relationships.



Summary of Neural Networks



Weighted Sums & Activations

Neural networks perform a sequence of weighted sums followed by non-linear activation functions.



Optimisable Parameters

Each weight and bias is a parameter optimised iteratively using gradient descent to model training data.



Universal Approximation Theorem

A feedforward neural network with sufficient capacity can approximate any continuous function to arbitrary precision.

A Deeper Look Into the Math

Gradient Descent & The Chain Rule

Parameter Optimisation

Just like linear regression, we optimise model parameters by calculating gradients with respect to the loss function.

Nested Functions & Backpropagation

For multi-layer networks, computing these gradients efficiently across nested functions requires the **chain rule of calculus** which is implemented via **backpropagation**.

The Backprop Pipeline

STEP 1 Forward Pass

Compute predictions ($\hat{y}$) and loss L .



STEP 2 Backward Pass

Propagate error gradients backward from loss to weights using the chain rule.

Find out more on backprop here: <https://www.youtube.com/watch?v=llg3gGewQ5U&t>

Images as Inputs

Grayscale

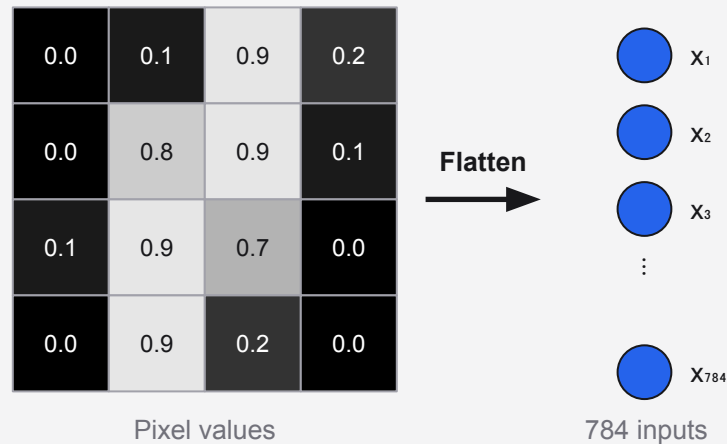
Each pixel is a number from 0 (black) to 1 (white). An MNIST digit is a 28×28 grid, so 784 values.

Colour (RGB)

Each pixel has three values, one each for red, green and blue, giving 3 channels.

Flattening

The grid is flattened into one vector, so each pixel feeds one input neuron.



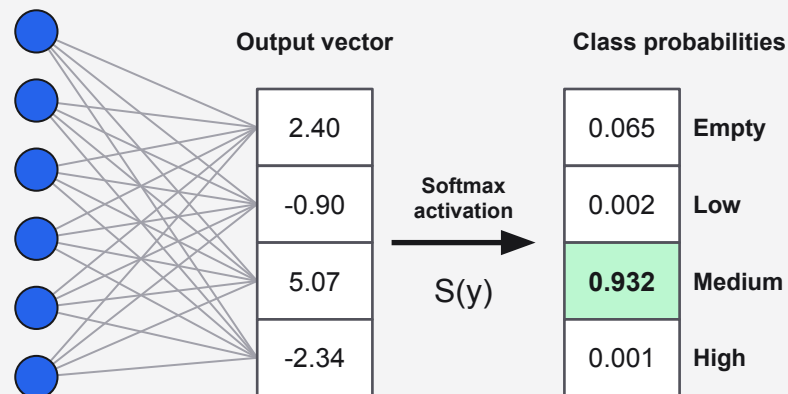
Classifiers + One Hot Encoding

Beyond Single Value Outputs

A lot of the time we don't just want a neural network to produce a value, we want to use it to classify.

Multiple Neurons & Softmax

To do this we have multiple neurons in the output layer (one per class) and use a function called Softmax to produce a probability distribution.



Neural Networks in Practice

3 Training Essentials

1. Network Architecture

The neural network model itself.

2. Loss Function

Measures prediction error against targets.

3. Optimiser

Algorithm used to update parameters.

In PyTorch

`optimizer.zero_grad()`

Resets existing gradients before backward pass.

`loss.backward()`

Performs backpropagation & computes gradients.

`optimizer.step()`

Updates network parameters using calculated gradients.

Questions & Discussion

Ready to dive deeper into hands-on implementation and training loops.